IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Fiche de synthèse

Jour 4 — Réseaux de neurones

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

1 Le neurone artificiel

Le neurone artificiel est une version enrichie du perceptron : il remplace la fonction signe par une **fonction d'activation** non linéaire.

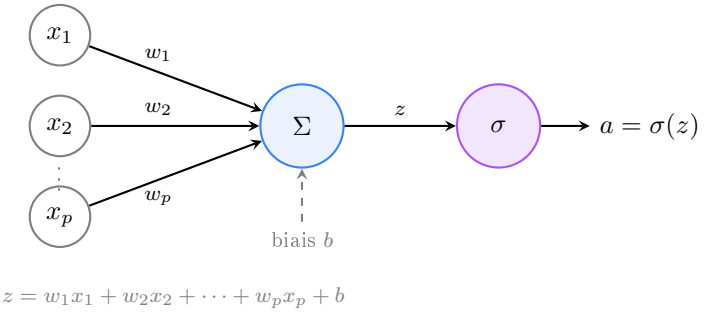
Neurone artificiel

Un neurone calcule deux étapes :

$$\text{Somme pondérée :} \quad z = \mathbf{w}^T\mathbf{x} + b \tag{1}$$

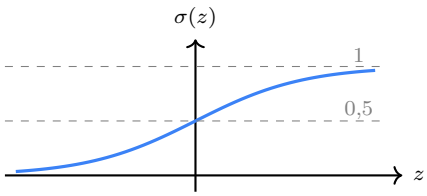
$$\text{Activation :} \quad a = \sigma(z) \tag{2}$$

où σ est une fonction d'activation non linéaire.

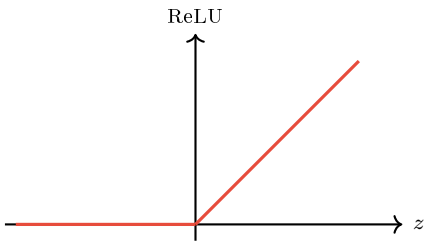


2 Fonctions d'activation

Fonction	Formule	Propriétés
Sigmoïde	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	Sortie dans $(0,1)$. Idéale pour les probabilités. Dérivée : $\sigma'(z) = \sigma(z)(1-\sigma(z))$.
ReLU	$\mathrm{ReLU}(z) = \max(0,z)$	Sortie ≥ 0. Rapide à calculer. La plus utilisée dans les réseaux modernes.
Signe	$\mathrm{signe}(z) \in \{0,1\}$	Échelon binaire (perceptron Jour 3). Non différentiable $\Rightarrow$ pas de gradient.



Sigmoïde



ReLU

Pourquoi les activations sont indispensables

Sans fonction d'activation, un empilement de couches linéaires reste **linéaire**. Les activations introduisent la **non-linéarité** qui permet au réseau de modéliser des relations complexes (comme le XOR).

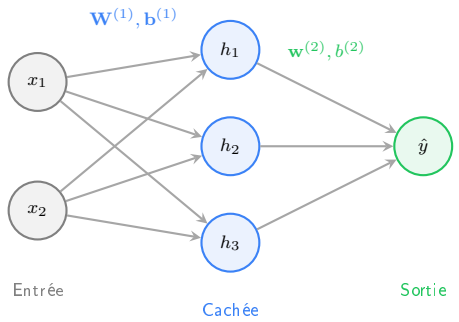
3 Réseau multicouche (MLP)

Perceptron multicouche

Un **MLP** (*Multi-Layer Perceptron*) est composé de :

- **Couche d'entrée** : reçoit les features $\mathbf{x}$ (pas de calcul)
- **Couche(s) cachée(s)** : neurones qui créent de nouvelles représentations
- **Couche de sortie** : produit la prédiction $\hat{y}$

Chaque neurone est connecté à **tous** les neurones de la couche suivante.



3.1 Forward pass (propagation avant)

Le calcul se fait **couche par couche**, de l'entrée vers la sortie :

$$\text{Couche cachée :} \quad \mathbf{h} = \sigma\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big) \tag{3}$$

$$\text{Couche de sortie :} \quad \hat{y} = \sigma\big(\mathbf{w}^{(2)T}\mathbf{h} + b^{(2)}\big) \tag{4}$$

Réseau 2-2-1 — calcul complet

Entrée $\mathbf{x} = (5, 4)^T$:

$z_1 = 0{,}3 \times 5 + 0{,}5 \times 4 - 2 = 1{,}5 \;\Rightarrow\; h_1 = \sigma(1{,}5) = 0{,}82$
$z_2 = -0{,}2 \times 5 + 0{,}4 \times 4 + 0{,}5 = 1{,}1 \;\Rightarrow\; h_2 = \sigma(1{,}1) = 0{,}75$
$z_{\text{out}} = 0{,}8 \times 0{,}82 + 0{,}6 \times 0{,}75 - 0{,}5 = 0{,}61 \;\Rightarrow\; \hat{y} = \sigma(0{,}61) = 0{,}65$

Prédiction : 65% de probabilité d'être en classe 1.

4 Apprentissage et backpropagation

4.1 Fonctions de perte

Tâche	Perte	Usage
Régression	$J = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$	MSE (Jours 1–2)
Classification	$L = -\frac{1}{n}\sum\big[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\big]$	Cross-entropy binaire

4.2 Backpropagation (intuition)

Propager l'erreur en arrière

La **backpropagation** calcule la contribution de **chaque poids** à l'erreur finale, en remontant de la sortie vers l'entrée. C'est la **règle de la chaîne** (Jour 2) appliquée à chaque couche du réseau.

La boucle d'entraînement

L'entraînement d'un réseau répète 3 étapes :

1. **Forward pass** : calculer $\hat{y}$ couche par couche
2. **Backward pass** : calculer ∇L pour chaque poids (backpropagation)
3. **Mise à jour** : $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla L$

C'est la **même boucle** qu'au Jour 2, mais appliquée à toutes les couches.

5 Comparatif : perceptron vs réseau

	Perceptron (Jour 3)	**Réseau MLP (Jour 4)**
Architecture	1 couche (entrée → sortie)	Plusieurs couches
Activation	Signe (0 ou 1)	Sigmoïde, ReLU, ...
Frontière	Linéaire (droite)	**Non linéaire** (courbe)
Résout XOR ?	Non	**Oui**
Accuracy dataset	∼87–90%	∼93–97%
Apprentissage	Correction d'erreurs	Backpropagation + GD
scikit-learn	`Perceptron`	`MLPClassifier`

6 Limites et risques

Surapprentissage (overfitting)

Un réseau trop gros peut **mémoriser** les données au lieu de **généraliser**. Solutions : jeu de **validation**, **régularisation** (L_2), **dropout**, **early stopping**.

Biais dans les données

Un réseau ne fait qu'apprendre les **patterns présents dans les données**. Si les données contiennent des biais (discrimination), le modèle les reproduira. La responsabilité éthique incombe aux humains qui conçoivent le système.

7 Formules essentielles à retenir

Formules du Jour 4

$$\text{Neurone} \quad a = \sigma(\mathbf{w}^T\mathbf{x} + b) \tag{5}$$

$$\text{Sigmoïde} \quad \sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{6}$$

$$\text{ReLU} \quad \mathrm{ReLU}(z) = \max(0, z) \tag{7}$$

$$\text{Forward (cachée)} \quad \mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \tag{8}$$

$$\text{Forward (sortie)} \quad \hat{y} = \sigma(\mathbf{w}^{(2)T}\mathbf{h} + b^{(2)}) \tag{9}$$

$$\text{Cross-entropy} \quad L = -\frac{1}{n}\sum\left[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\right] \tag{10}$$

$$\text{Mise à jour} \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\nabla L \tag{11}$$

Pipeline du Jour 4 : $\mathbf{x} \xrightarrow{\mathbf{W}^{(1)}} \sigma \rightarrow \mathbf{h} \xrightarrow{\mathbf{w}^{(2)}} \sigma \rightarrow \hat{y}$

Les 4 jours en un mot : J1 Représentation → J2 Optimisation → J3 Classification → J4 Réseaux

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0